Run-time polymorphism using virtual function

Virtual function

- Virtual function is used to support run-time polymorphism
- Two types of polymorphism
- Static: at compile time e.g. function overloading and operator overloading
- Dynamic : at run-time through the use of virtual functions
- Foundation of virtual function are pointers to derived classes

Pointers to derived classes

- base *p;
- base base_ob;
- derived d_ob;
- p=&base_ob;
- p=&d_ob;
- Base class pointer can point to an object of any class derived from that base without generating a type mismatch error. Known as up casting
- But you can access only those members of the derived object that ware inherited from the base. Because the base pointer has knowledge only of the base class, knows nothing about derived class members
- But derived class pointer cannot point to base class. Known as Down casting is not allowed

Example

```
 class base
{
  int x;
  public:
   void setx(int i){x=i;}
   int getx(){return x;}
};
 class derived: public base
    {
   int y;
   public:
  void sety(int i){y=i;}
  int gety(){return y;}
};
```

```
 int main()
 {
    base *p;
    base b_ob;
    derived d_ob;
   p=&b_ob;
  p->setx(10);
  cout<<"base x "<<p->getx()<<endl;
 p=&d_ob;
p->setx(99);
d_ob.sety(88); // p->sety(55);//give error
cout<<"Base object x="<<p->getx()<<endl;
 cout<<" derived object y"<<d_ob.gety()<<endl; return 0;}
```

Base class function overridden in derived class

```
class base
{
  public:
   int i;
    base(int x){i=x;}
     void func()
   {
     cout<<"base version\n";
     cout<<i<<endl;}
  };
```

```
class derived1:public base
{
  public:
   derived1(int x): base(x){}
   void func()
    {
    cout<<" derived 1
version\n";
   cout<<i*i<<endl;
  }
};
```

```
int main()
   {
        base *p;
    derived1 ob(10);
    p=&ob;
    p->func();
}
```

Virtual function

- A virtual function is a member function that is declared within base class and redefined by a derived class
- To declare virtual function precede the function declaration with keyword virtual
- When a class containing virtual function is inherited, the derived class redefines the virtual function relative to derived class
- In essence virtual function implement one interface multiple methods philosophy that underlies polymorphism

Virtual function

- How support run-time polymorphism
- Consider many derived classes implemented virtual function
- Base class pointer pointing to derived class object that contains virtual function and that virtual function is called
- C++ determines which version of that function will be executed based upon the type of object being pointed to by the pointer and this determination is made at run time so it is called dynamic or run time polymorphism . Also known as late binding
- A class that contains a virtual function is referred to as a polymorphic class

Example

```
 class base
{
  public:
   int i;
    base(int x){i=x;}
    virtual void func()
   {
     cout<<“base version:”;
     cout<<i<<endl;}
   };
```

```
 class derived1:public base
{
  public:
   derived1(int x): base(x){}
   void func()
    {
    cout<<“ derived 1 version:”;
   cout<<i*i<<endl;
  }
};
class derived2:public base
{
  public:
   derived2(int x): base(x){}
   void func()
    {
    cout<<“ derived 2 version:”;
   cout<<i+i<<endl;
  }};
```

```
 int main()
  {
   base *p;
   base ob(10);
   derived1 d_ob1(10);
   derived2 d_ob2(10);
   p=&ob;
  p->func();    //use base class func
  p=&d_ob1;
  p->func();  //derived1 func()
  p=&d_ob2;
  p->func();  // derived2 func();
 return 0;
}
```

output

base version:10

derived 1 version:100

derived 2 version:20

Virtual function is redefined in derived classes referred as function overriding

Type of the object being pointed to determines which version of an overridden virtual function will be executed when accessed via a base class pointer and this decision is made at run time

Run time polymorphism is achieved only when virtual function is accessed through a pointer to the base class or reference

Example

```
class base
{
 public:
  int i;
   base(int x){i=x;}
   virtual void func()
  {
    cout<<“base version:”;
    cout<<i<<endl;}
 };
```

```
class derived1:public base
{
  public:
   derived1(int x): base(x){}
   void func()
    {
     cout<<“ derived 1 version:”;
   cout<<i*i<<endl;
  }
};
class derived2:public base
{
  public:
   derived2(int x): base(x){}
};
```

```
int main()
 {
  base *p;
  base ob(10);
  derived1 d_ob1(10);
  derived2 d_ob2(10);
  p=&ob;
 p->func();    //use base class
func
 p=&d_ob1;
 p->func();  //derived1 func()
 p=&d_ob2;
 p->func();  // base version
func();
 return 0;
}
```

discussion

- Virtual functions are hierarchical in order of inheritance
- When derived class does not override a virtual function the function defined within its base class is used

Virtual functions are hierarchical in order of inheritance

```
class base
{
  public:
   int i;
    base(int x){i=x;}
    virtual  void func()
   {
     cout<<"base version\n";
     cout<<i<<endl;}
  };
class derived1:public base
{
  public:
   derived1(int x): base(x){}
};
```

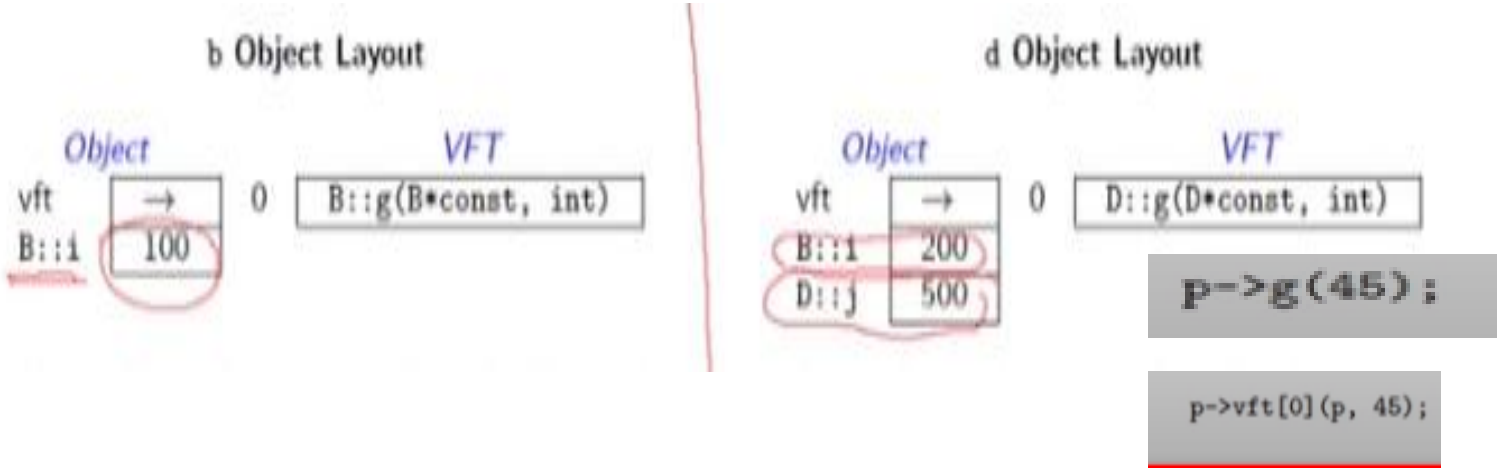
Virtual function can respond to random events that occur at run time

```
class derived2: public derived1
{
   public:
   derived2(int x): derived1(x){}
   void func()
    {
    cout<<" derived 2 version\n";
   cout<<i*i<<endl;
  }
  };
int main()
   {
base *p;
    derived2 ob(10);
    p=&ob;
    p->func();
   derived1 ob1(20);
   p=&ob1;
p->func();
}
```

```
int main()
 {
  base *p;
 derived1 d_ob1(10);
  derived2 d_ob2(10);
  int i,j;
  for(i=0;i<10;i++)
  {
   j=rand();
   if(j%2)    // if odd
   p=&d_ob1;
  else
  p=&d_ob2;
p->func();
 return 0;
}
```

Virtual function table(VFT)

Base class has virtual function g
Base class object b and derived class object d
b *p
p=&b;
p->g(45) converted to p->vft[0](p,45)
P=&d;
 p->g(46); converted to p->vft[0](p,46)



Virtual function table

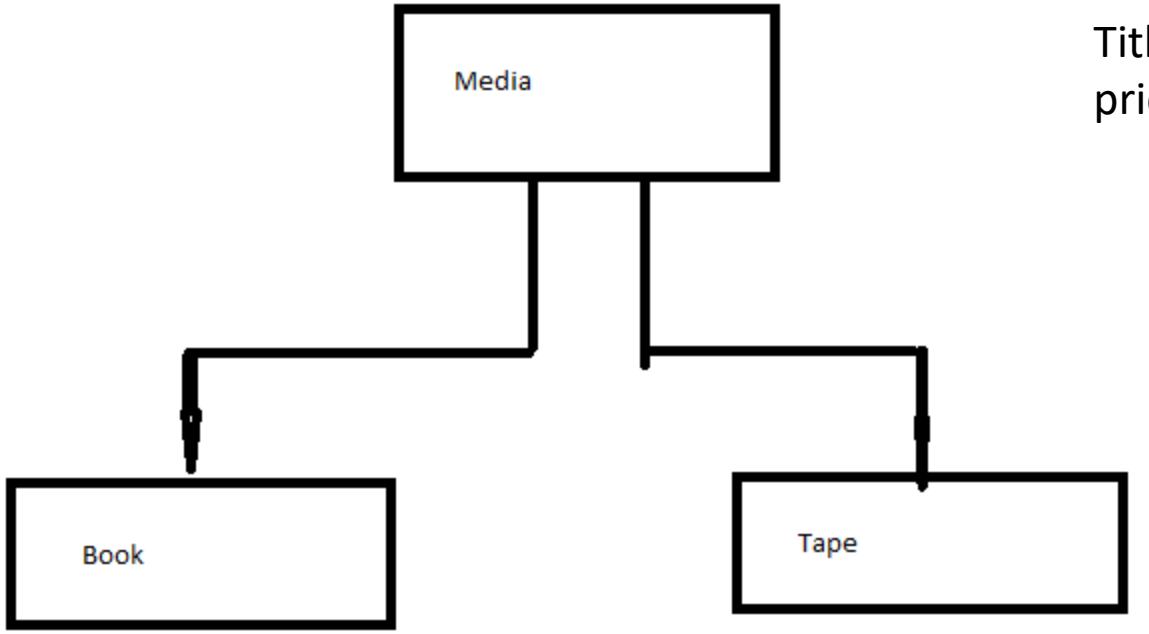
- Whenever a class defines a virtual function a hidden member variable is added to the class which points to an array of pointers to (virtual) functions called the **Virtual Function Table (VFT)**.
- VFT pointers are used at run-time to invoke the appropriate function implementations, because at compile time it may not yet be known if the base function is to be called or a derived one implemented by a class that inherits from the base class.
- VFT is class-specific – all instances of the class have the same VFT.
- VFT carries the **Run-Time Type Information (RTTI)** of objects.

```
#include <iostream>
using namespace std;

class B {
public:
    void f() {
        cout << "B::f()" << endl;
    }
    virtual void g() {
        cout << "B::g()" << endl;
    }
};

class D : public B {
public:
    void f() {
        cout << "D::f()" << endl;
    }
    virtual void g() {
        cout << "D::g()" << endl;
    }
};
```

```
int main() {
    B b;
    D d;

    B* pb = &b;
    B* pd = &d;   // Upcast

    B& rb = b;
    B& rd = d;    // Upcast

    // Direct object calls
    b.f();   // B::f()
    b.g();   // B::g()
    d.f();   // D::f()
    d.g();   // D::g()
```

```
// Pointer calls
    pb->f();   // B::f() -- Static Binding
    pb->g();   // B::g() -- Dynamic Binding
    pd->f();   // B::f() -- Static Binding
    pd->g();   // D::g() -- Dynamic Binding

    // Reference calls
    rb.f();    // B::f() -- Static Binding
    rb.g();    // B::g() -- Dynamic Binding
    rd.f();    // B::f() -- Static Binding
    rd.g();    // D::g() -- Dynamic Binding

    return 0;
}
```

Example



Title
price

Number
of pages

Playing
time

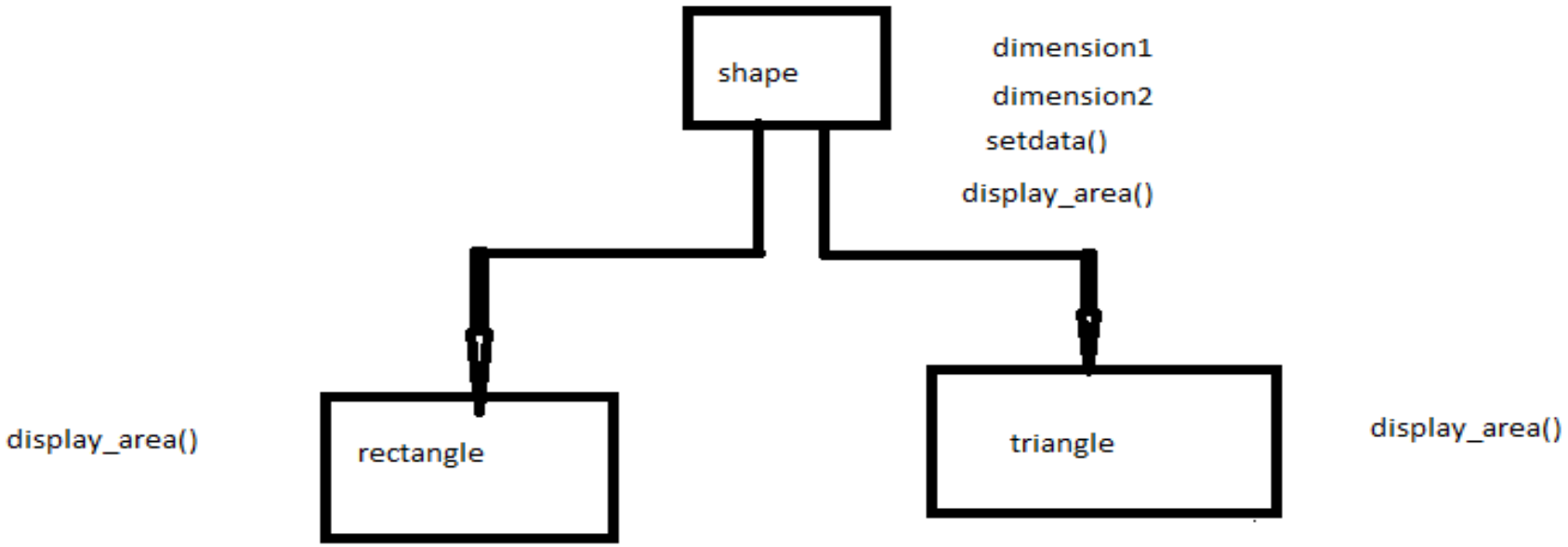
```
#include <iostream>
#include <cstring>
using namespace std;
class media
   {
     protected:
     char title[10];
     float price;
    public:
    media(char *s,float n)
    {
     strcpy( title, s);
    price=n;
   }
   virtual void display(){}};
```

```
 class book: public media
   {
    int pages;
    public:
    book(char *s,float n, int p):media(s,n)
   {
   pages=p;
}
  void display();
};
void book :: display()
  {
   cout<<“ title “<<title<<“price “<<price<<“
pages “<<pages<<endl;
}
```

```
class tape:public media
 {
  float time;
 public:
  tape(char *s, int n, int t):media(s,n)
 {
  time=t;
 }
void display();
};
 void tape :: display()
 {
  cout<<“ title “<<title<<“price “<<price<<“ time
“<<time<<endl;
}
```

```
int main()
 {
  char *t=new char[10];
 float price,time;
int p;
 cout<<"enter book title, price and pages\n";
 cin>>t>>price>>p;
 book book1(t,price,p);
cout<<"enter media title, price and time\n";
 cin>>t>>price>>time;
 tape tp(t,price,time);
media *list;
 list=&book1;
 list->display();
 list=&tp;
 list->display();
 return 0;
}
```

example


shape
dimension1
dimension2
setdata()
display_area()
rectangle
display_area()
triangle
display_area()

```
class shape
{
  double dim1;
 double dim2;
public:
 void setdimension(double d1,double d2)
   {
   dim1=d1;
 dim2=d2;
}
void getdim(double & d1,double &d2)
  {
   d1=dim1;
  d2=dim2;
}
virtual void display_area()=0;
};
```

```
class rectangle:public shape
{
  public:
 void display_area()
  {
 double d1,d2;
 getdim(d1,d2);
cout<<"rectangle area= "<<d1*d2<<endl;
}
};
```

```
class triangle:public shape
{
 public:
void display_area()
{
  double d1,d2;
getdim(d1,d2);
cout<<" triangle area= "<<0.5*d1*d2<<endl;
}
};
```

```
int main()
{
 shape *s;
rectangle r;
triangle t;
r.setdimension(3.3,4.5);
t.setdimension(4.0,5.0);
s=&r;
s->display_area();
s=&t;
s->display_area();
return 0;
}
```

Rules for virtual functions

1. Virtual function must be member of some class
2. They cannot be static members
3. They are accessed by using object pointers
4. A virtual function can be friend of other class
5. A virtual function in a base class must be defined even though it may not be used
6. The prototypes of the base class version of a virtual function and all the derived class versions must be identical. If two functions with the same name but different prototypes considers them as overloaded functions and virtual function mechanism is ignored
7. We cannot have virtual constructor but we can have virtual destructors
8. Base pointer can point to any derived object but reverse is not true
9. When a base pointer pointing to derived class , incrementing or decrementing it will not make to point to next object of the derived class but incremented or decremented relative to base class
10. If a virtual function defined in base class , it need not be necessarily redefined in derived class. In such cases it will invoke the base function

Pure virtual function

- Virtual function declared in base class has no meaningful operation to perform
- So c++ support pure virtual function
- Has a signature but no body. But if yow want it can have body also

General form

```
virtual type functionname(parameters)=0;
```

It forces derived classes to override it. If not then compiler time error results

Abstract base class

- A class containing at least one **Pure Virtual Function** is called an **Abstract Base Class**.
- **Pure Virtual Functions** may be inherited or defined in the class.
- No instance can be created for an **Abstract Base Class**.
- Naturally, it does not have a constructor or a destructor.
- An **Abstract Base Class**, however, may have other virtual (non-pure) and non-virtual member functions as well as data members.
- Data members in an **Abstract Base Class** should be protected. Of course, private and public data are also allowed.
- Member functions in an **Abstract Base Class** should be public. Of course, private and protected methods are also allowed.
- A **Concrete Class** must override and implement all **Pure Virtual Functions** so that it can be instantiated.

Abstract base class

- You can still create pointer of an abstract class
- It is also possible to have a reference to an abstract class
- The purpose of an abstract class is to define a common protocol for a set of concrete subclasses. This is useful when defining objects that share code, abstract ideas, etc. Classes that can be used to instantiate objects are called **concrete classes**.
- The purpose of an **abstract class** is to provide an appropriate base class from which other classes can inherit.

Pure virtual function

```
 class B
{
   public:
   virtual void example()=0;
   };
class C: public B
{
 public:
   void example()
{
  cout<<“ c book written by balaguruswami\n”;
}
Class CPP:public B
{
public:
 void example()
{
  cout<<“c++ book written by balaguruswamy\n”;}};
```

```
int main()
 {
   B *arr[2];
    C e1;
  CPP e2;
  arr[0]=&e1;
  arr[1]=&e2;
  arr[0]->example();
  arr[1]->example();
return 0;}
```

Virtual destructor

constructor cannot be virtual because

1. to create an object the constructor of the object class must be of the same type as the class. But this is not possible with a virtually implemented constructor

2. At the time of calling a constructor the virtual table would not have been created to resolve any virtual function calls

Virtual destructor

```
class A
{
  public:
 ~A()
{
cout<<“base class destructor\n”;
}
};
 class B : public A
{
  public:
~B()
{
cout<<“derived destructor\n”;
}};
```

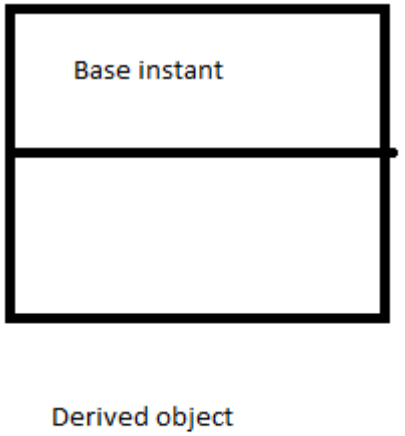
```
int main()
 {
  A *ptr=new B();
  delete ptr;
}
```

Virtual destructor

It will trigger the base class destructor, derived class destructor will not be called. This may lead to a memory leak situation

So we have to write

```
  virtual ~A()
{
cout<<“base class destructor\n”;
}
```



Now it called derived destructor then base class destructor

Output of following program

```
#include<iostream>
using namespace std;

class Base
{
public:
   virtual void show() { cout<<" In Base n";
}
};

class Derived: public Base
{
public:
   void show() { cout<<"In Derived n"; }
};

int main(void)
{
   Base *bp, b;
   Derived d;
   bp = &d;
   bp->show();
   bp = &b;
   bp->show();
   return 0;
}
```

```
In Base
In Base
```

```
In Base
In Derived
```

```
In Derived
In Derived
```

```
In Derived
In Base
```

Predict the output of following program.

```
#include<iostream>
using namespace std;
class Base
{
public:
   virtual void show() = 0;
};

class Derived : public Base { };

int main(void)
{
   Derived q;
   return 0;
}
```

Choose option

A Compiler error: derived is abstract

B Compiler error: there cannot be empty derived class

```
#include<iostream>
using namespace std;
 class Base
{
public:
    virtual void show() = 0;
};

class Derived: public Base
{
public:
    void show() { cout<<"In Derived n"; }
};
 int main(void)
{
    Derived d;
    Base &br = d;
    br.show();
    return 0;}
```

Compiler Error in line "Base &br = d;"

Empty Output

In Derived

```
#include<iostream>
using namespace std;

class Base
{
public:
   virtual void show() { cout<<" In Base n"; }
};

class Derived: public Base
{
public:
   void show() { cout<<"In Derived n"; }
};

int main(void)
{
   Base *bp = new Derived;
   bp->show();

   Base &br = *bp;
   br.show();

   return 0; }
```

```
In Base
In Base
```

```
In Base
In Derived
```

```
In Derived
In Derived
```

```
In Derived
In Base
```

Comparison between early and late binding

- **Early binding**

Advantages : very efficient

Disadvantages : lack of flexibility

- **Late binding**

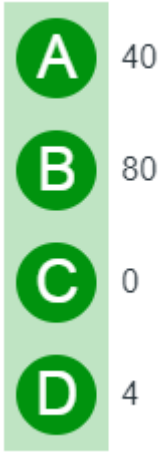
Advantages : flexibility

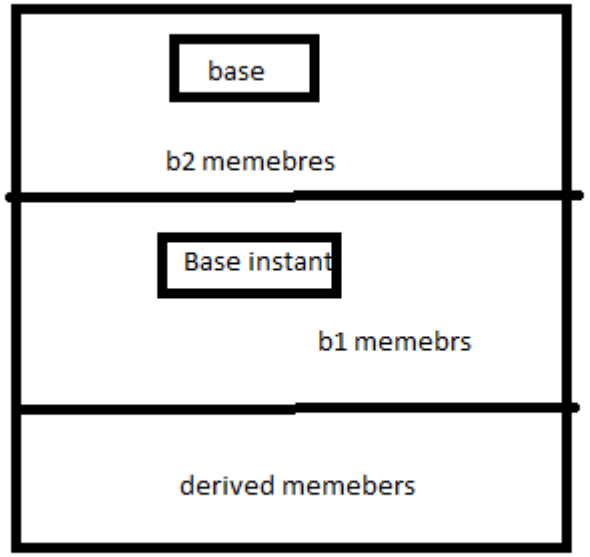
Disadvantages : more overhead associated with function call so makes such call slower than early binding

Assume that an integer takes 4 bytes and there is no alignment in following classes, predict the output.

```
#include<iostream>
using namespace std;
 class base {
   int arr[10];
};
 class b1:  public base { };
 class b2:  public base { };
 class derived: public b1, public b2 {};
 int main(void)
{
  cout << sizeof(derived);
  return 0; }
```

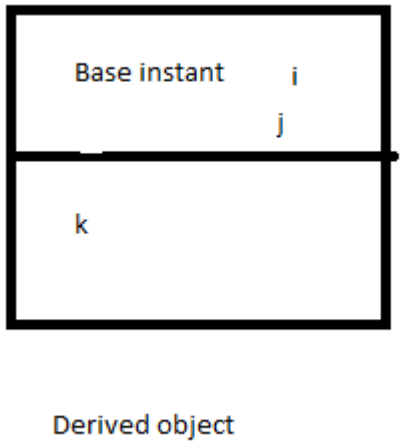
B





```
#include <iostream>
using namespace std;
class base
{
  int i;
  float j;


};
class derived:public base
{
  int k;
};

int main()
{
   base ob;
   derived ob1;
cout<<sizeof(ob)<<endl;     12 bytes
   cout<<sizeof(ob1)<<endl;   16 bytes
   return 0; }
```

Integer=4
Float =8
bytes



```
#include<iostream>

using namespace std;
class P {
public:
   void print()  { cout <<" Inside P"; }
};

class Q : public P {
public:
   void print() { cout <<" Inside Q"; }
};

class R: public Q { };

int main(void)
{
  R r;
  r.print();
  return 0;
}
```

```
include<iostream>
using namespace std;

class Base {
private:
    int i, j;
public:
   Base(int _i = 0, int _j = 0): i(_i), j(_j) { }
};
class Derived: public Base {
public:
    void show(){
      cout<<" i = "<<i<<"  j = "<<j;
    }
};
int main(void) {
  Derived d;
  d.show();
  return 0;
}
```

i = 0 j = 0

Compiler Error: i and j are private in Base

Compiler Error: Could not call constructor of Base

```
#include<iostream>
using namespace std;

class Base
{
public:
   int fun()  { cout << "Base::fun() called"; }
   int fun(int i)  { cout << "Base::fun(int i) called"; }
};

class Derived: public Base
{
public:
   int fun() {  cout << "Derived::fun() called"; }
};

int main()
{
   Derived d;
   d.fun(5);
   return 0;
}
```

Base::fun(int i) called

Derived::fun() called

Base::fun() called

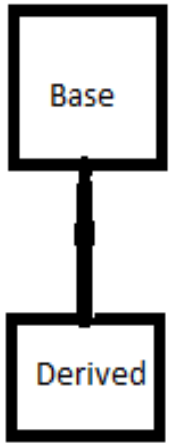
Compiler Error

If a derived class writes its own method, then all functions of base class with same name become hidden, even if signaures of base class functions are different. In the above question, when fun() is rewritten in Derived, it hides both fun() and fun(int) of base class.

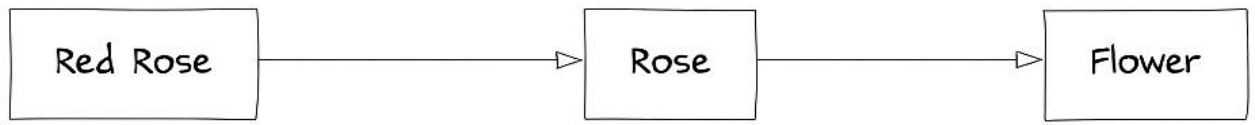
You have to write **Base::fun;** in derived class

- We often find one object is a *specialization* / *generalization* of another
- OOAD models this using **ISA** relationship
- C++ models **ISA** relationship by *Inheritance* of classes

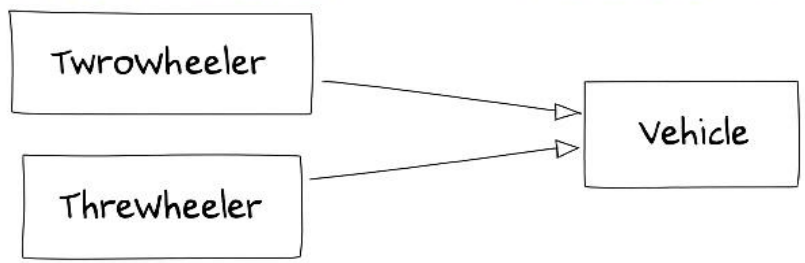
UML diagram
Unified modeling language
ISA relationship
Rose is a flower



- Rose **ISA** Flower
  - Rose has the properties of Rose – like rosy fragrance,
  - Rose has some additional properties – like its red
  - Rose is a *gnecialization* of Rose
  - Flower is a *generalization* of Rose
- Red Rose **ISA** Rose
  - Red Rose has the properties of Rose – like rosy fragrance
  - Red Rose has some additional properties – like it is red
  - Rose is a *generalization* of Red Rose



- TwoWheeler **ISA** Vehicle; ThreeeWheeler **ISA** Vehicle



- Manager **ISA** Employee

homework

1. Declare a base class A and three derived classes called D1,D2 and D3. show use of dynamic polymorphism for these classes having overriding function display. Define constructor for all classes